Conductor Framework — Layer Contracts

Productization baseline · Forked 2026-06-04
Source: /home/unctad/projects/conductor-framework
Canonical seam names from: docs/architecture/seams.md · docs/architecture/system.md

Table of Contents

1. Architecture Overview
2. Layer 3 — Conductor
3. Layer 2 — Domain Orchestrators
   - 3.1 PhaseOrchestrator
   - 3.2 ExecutionOrchestrator & OutputChannel
   - 3.3 FindingsOrchestrator
   - 3.4 ChainOrchestrator
   - 3.5 SessionOrchestrator
   - 3.6 FilterOrchestrator
4. Layer 1 — Engines
   - 4.1 Engine ABC & EngineCapabilities
   - 4.2 Representative Engine: CredentialBruteEngine
5. Support Layer — Agent Loop
   - 5.1 AgentState
   - 5.2 GraphResolver
   - 5.3 LeafEvaluator
   - 5.4 Planner
6. Vault Pipeline
   - 6.1 QwenClient
   - 6.2 ContextBuilder
   - 6.3 VaultWriter
   - 6.4 VaultEmbedder
7. Export Layer
   - 7.1 VaultManager
   - 7.2 ExportService (investigation_core)
   - 7.3 ExporterService (exporters)
8. Seam Inventory
9. Input / Output Summary
10. Database Map

1. Architecture Overview

The conductor-framework implements a three-level orchestration architecture:

```
┌──────────────────────────────────────────────────────────────┐
│  LAYER 3 — Conductor                                         │
│  conductor.py · _InvestigationContext                        │
│  Manages investigation lifecycle, wires all orchestrators    │
└──────────────────────────┬───────────────────────────────────┘
                           │ wires 6 orchestrators
┌──────────────────────────▼───────────────────────────────────┐
│  LAYER 2 — Domain Orchestrators                              │
│  Phase · Execution · Findings · Chain · Session · Filter     │
│  Coordinate investigation phases; enforce seam contracts     │
└──────────────────────────┬───────────────────────────────────┘
                           │ dispatches via OutputChannel
┌──────────────────────────▼───────────────────────────────────┐
│  LAYER 1 — Stateless Engines                                 │
│  apps/conductor/engines/*.py  (9 exploitation + recon/enum)  │
│  Run(target, config, session, channel) → EngineResult        │
└──────────────────────────────────────────────────────────────┘

Cross-cutting:
  Support Layer  →  AgentState · GraphResolver · LeafEvaluator · Planner
  Vault Pipeline →  QwenClient · ContextBuilder · VaultWriter · VaultEmbedder
  Export Layer   →  VaultManager · ExportService · ExporterService
```

Phase sequence: recon → enumeration → exploitation

Three-level rule: No fourth orchestration level without explicit approval.

Operator drives: Phase completions are operator-triggered, never auto-complete.

---

2. Layer 3 — Conductor

File: apps/conductor/conductor.py

Class: `Conductor`

The top-level coordinator. Creates one _InvestigationContext per investigation, wires all six domain orchestrators, and manages state persistence in data/orchestration.db.

Constructor

```
async def __init__(
    bus: EventBus | None = None,
    db_path: Path | None = None,
) -> None
```

Parameter	Type	Default	Notes
bus	`EventBus \	None`	defaultbus
db_path	`Path \	None`	data/orchestration.db

Public Methods

```
async def start_investigation(
    target: str,
    config: dict,
    storage_bridge=None,
    investigation_id: str | None = None,
) -> str
```

Inputs
Output
Writes
Seams
Side-effect

```
async def cancel_investigation(investigation_id: str) -> None
async def get_investigation_status(investigation_id: str) -> dict
async def list_active_investigations() -> list[dict]
async def operator_complete_exploitation(investigation_id: str) -> bool
def refresh_target_auth(target_base: str, auth: dict) -> int
def get_context(investigation_id: str) -> _InvestigationContext | None
```

Investigation State Machine

```
start_investigation()
  │
  ▼  CREATED
_run_phases()
  │
  ├─ RECON_RUNNING  → phase_orch.start_next_phase() → engines dispatched
  │                   wait phase_completed event (timeout 300s)
  │
  ├─ ENUM_RUNNING   → same as above
  │                   AFTER enumeration: filter_orch.run_filter()
  │
  ├─ EXPLOIT_RUNNING → operator-driven completion only (Seam C3)
  │
  └─ COMPLETED / FAILED / CANCELLED
```

Internal: `_InvestigationContext`

Responsibility: Runtime container — holds all six orchestrators for one active investigation.

Orchestrators instantiated: SessionOrchestrator, FindingsOrchestrator, ChainOrchestrator, ExecutionOrchestrator, PhaseOrchestrator, FilterOrchestrator.

Wiring graph (constructor, lines 101–136):

```
findings_orch ──wire_chain──────────────► chain_orch
chain_orch    ──wire_execution──────────► exec_orch
chain_orch    ──wire_session────────────► session_orch
exec_orch     ──wire_findings───────────► findings_orch
exec_orch     ──wire_session────────────► session_orch
exec_orch     ──wire_phase──────────────► phase_orch
phase_orch    ──wire_execution──────────► exec_orch
filter_orch   ──wire_findings───────────► findings_orch
filter_orch   ──wire_reinvocation_dispatcher()
filter_orch   ──wire_db_writer()        → validation.db  ← Seam 3
filter_orch   ──wire_block_reason_writer() → validation.db
```

Seam 3 closure (lazy DB_PATH import, inside constructor closure):

- Target: validation.db via NormalizedFindingRepository
- Guard: import is inside the closure body to prevent accidental orchestration.db capture

---

3. Layer 2 — Domain Orchestrators

3.1 PhaseOrchestrator

File: apps/conductor/orchestrators/phase.py

```
async def start_next_phase() -> str | None
def current_phase(investigation_id: str) -> str
async def operator_complete_exploitation() -> None
```

Method	Input	Output	Seam
startnextpha	—	phase name or None	Seam 19
current_phas	investigation_id	`"recon" \	"enumeration" \
operatorcom	—	—	Seam C3

Phase sequence constant: PHASESEQUENCE = ["recon", "enumeration", "exploitation"]

Events emitted: phasestarted, phasecompleted, phase_failed

3.2 ExecutionOrchestrator & OutputChannel

File: apps/conductor/orchestrators/execution.py

```
async def dispatch_phase(phase: str) -> None
async def dispatch_engine(engine_id: str, extra_config: dict | None = None) -> str
async def cancel_all(investigation_id: str) -> None
def running_engines(investigation_id: str) -> list[dict]
```

Watchdog thresholds (Seam 9):

resource_cost	Threshold
"light"	30 s
"medium"	120 s
"heavy"	600 s

OutputChannel

Constructed before each engine.run() call. The single ingress point for all engine output (Seam 8).

```
# Construction (inside ExecutionOrchestrator._launch)
channel = OutputChannel(
    investigation_id=investigation_id,
    engine_id=engine_id,
    findings_orch=self._findings_orch,
    session_orch=self._session_orch,
    branch_writer=self._branch_writer,
)
```

OutputChannel public methods:

Method	Input	Effect	Seam
emit_finding(	finding dict	→ FindingsOrchestrator.ingest()	Seam 8
record_reque	URL	→ SessionOrchestrator.record_request()	—
recordrespon	URL, int	429 detection + cooldown	—
createbranch	—	Creates exploitation_branches record	Seam 13
emitstage(br	—	Updates branch state + stage_log	Seam 13
storeauth(inv	investigation_id, dict	→ SessionOrchestrator.store_auth()	Seam 20

Branch state machine (emit_stage, Seam 13 — unidirectional):

```
pending → armed → running → succeeded
                          → exhausted
                          → failed
                          → cancelled
```

3.3 FindingsOrchestrator

File: apps/conductor/orchestrators/findings.py

```
async def ingest(finding: dict, engine_id: str) -> str | None
```

Input
Output
Algorithm
Events
Seam 8

Finding dict required keys: id (fndxxx), category, title, severity, evidence, probeid, phase

---

3.4 ChainOrchestrator

File: apps/conductor/orchestrators/chain.py

```
async def evaluate_finding(investigation_id: str, finding: dict) -> None
async def operator_arm(investigation_id: str, action_id: str) -> None
def chain_depth(investigation_id: str, branch_id: str) -> int
```

Chain depth limits: linear depth ≤ 5; per-engine dispatch count ≤ 3 (cycle guard).

Events emitted: chainautofired, chaindepthlimitreached, chainoperatorarmpending, chain_edge

---

3.5 SessionOrchestrator

File: apps/conductor/orchestrators/session.py

```
def get_auth(investigation_id: str) -> dict | None
def can_dispatch(target, engine_capabilities) -> tuple[bool, float]
def record_request(target: str) -> None
def record_response(target: str, status_code: int) -> None
def store_target_auth(target_base: str, auth: dict) -> None
def store_auth(investigation_id: str, auth: dict) -> None
```

Seam 15: Per-investigation auth store. No process-level store. INVESTIGATIONIDCTX context var used by ACTIVE-LEGACY probes.

3.6 FilterOrchestrator

File: apps/conductor/orchestrators/filter.py

```
async def run_filter(investigation_id: str) -> None
```

Timing: Called by Conductor after enumeration completes — not after recon (Seam 3 invariant).

Two-pass algorithm:

1. REGISTERED_RULES — drop / validate / incomplete, first-match-wins per finding
2. REINVOCATION_RULES — collect results, deduplicate, dispatch engines

Wired callbacks (set in InvestigationContext.init_):

```
wire_db_writer(async callable(finding_id, status, reason, gaps_json, validated_at))
# → writes normalized_findings.validation_status  (validation.db, Seam 3)

wire_block_reason_writer(async callable(finding_id, reason))
# → writes normalized_findings.enumeration_block_reason  (validation.db)

wire_reinvocation_dispatcher(async callable(engine_id, config))
# → dispatches via ExecutionOrchestrator
```

---

4. Layer 1 — Engines

4.1 Engine ABC & EngineCapabilities

File: apps/conductor/engines/base.py

```
class Engine(ABC):
    @property
    @abstractmethod
    def capabilities(self) -> EngineCapabilities: ...

    @abstractmethod
    async def run(
        self,
        target: str,
        config: dict,
        session_store: SessionStore,
        output_channel: OutputChannel,
    ) -> EngineResult: ...

    @abstractmethod
    async def cancel(self) -> None: ...

    @abstractmethod
    def health_check(self) -> EngineHealth: ...
```

EngineCapabilities (frozen dataclass)

Field	Type	Values
phases	tuple[str, ...]	("recon",) · ("enumeration",) · ("exploitation",)
finding_types	tuple[str, ...]	e.g. ("loginformdetected", "api_exposure")
target_types	tuple[str, ...]	("http",) · ("tcp",) · ("any",)
resource_cost	str	"light" · "medium" · "heavy"
concurrencysafewith	tuple[str, ...]	engine_ids safe to co-run
mutex_with	tuple[str, ...]	engine_ids that must not co-run

EngineResult

```
@dataclass
class EngineResult:
    status: str          # "success" | "failed" | "cancelled"
    findings_count: int
    error: str | None
```

Engine Conformance Rules

1. Stateless across runs — no instance variables persist between invocations
2. Auth read-only — never call login endpoints; read from session_store
3. Stream findings immediately — call outputchannel.emitfinding() as they emerge
4. Respect cancel() within 5 s — kill subprocesses, close sockets
5. Declare capabilities truthfully — concurrency and phase fields are enforced
6. Never create asyncio primitives in _init_ — bind event loop inside run()
7. Seam 16 — accept extendedpayloads and extendedwordlists kwargs in run()

V2 Engine Inventory

Engine	File	Phase	resource_cost
CredentialBru	credentialbruteengine.py	exploitation	medium
XssReflected	xssreflectedengine.py	exploitation	light
CmdiReflecte	cmdireflectedengine.py	exploitation	light
CmdiBlindTim	cmdiblindtime_engine.py	exploitation	medium
CmdiBlindBoo	cmdiblindboolean_engine.py	exploitation	medium
SqliAuthBypa	sqliauthbypass_engine.py	exploitation	medium
DefaultCreds	defaultcredsengine.py	exploitation	medium
AccessContro	accesscontrolengine.py	exploitation	medium
MetasploitRe	metasploitrecipeengine.py	exploitation	heavy
JSSpiderEngir	jsspiderengine.py	enumeration	light

4.2 Representative Engine: CredentialBruteEngine

File: apps/conductor/engines/credentialbruteengine.py

```
async def run(
    target: str,
    config: dict,
    session_store: SessionStore,
    output_channel: OutputChannel,
    *,
    extended_payloads: list | None = None,   # Seam 16 stub
    extended_wordlists: list | None = None,  # Seam 16 stub
) -> EngineResult
```

Key behaviors:

- Reads loginformdetected findings (validated/incomplete status only — Seam 12)
- Loads credential pairs from payload registry YAML
- Refreshes CSRF token per-request from finding.evidence["csrf_field"]

- On success: outputchannel.storeauth(investigationid, capturedsession) — Seam 20
- On exhaustion: outputchannel.emitstage("exhausted", payload={...}) → ExhaustionRecord — Seam 14

5. Support Layer — Agent Loop

5.1 AgentState

File: apps/conductor/agent_state.py

The persistent investigation reasoning state. Tracks facts, artifacts, pending work queue, and current position.

Schema

```
{
    "goal": {
        "objective": str,
        "success_conditions": list[str],
        "provided_self_user": str | None,
        "target_user": str | None,
    },
    "facts": list[str],
    "artifacts": {
        "credentials": list[dict],
        "sessions": list[dict],
        "tokens": list[dict],
        "files": list[dict],
    },
    "pending": list[{"concept": str, "priority": int}],
    "ruled_out": list[str],
    "completed": list[str],
    "current_position": {
        "authentication": "none" | "authenticated" | "admin",
        "privilege": "none" | "user" | "admin" | "root",
        "execution": "none" | "file_read" | "rce",
        "access_scope": list[str],
    },
    "solved": bool,
    "solved_reason": str,
    "planner_log": list[dict],
    "interrogation_attempts": int,
    "interrogation_log": list,
}
```

Persistence API

```
async def init_agent_state_async(investigation_id, target, objective) -> dict
async def load_agent_state_async(investigation_id: str) -> dict
async def save_agent_state_async(investigation_id: str, agent_state: dict) -> None
async def apply_verdict_to_state_async(
    investigation_id: str,
    verdict: Verdict,
    concept_id: str = "",
    engine_id: str = "",
    exec_state: str = "",
) -> None
```

applyverdicttostateasync:

- Concurrency-safe via per-investigation asyncio.Lock
- Updates: facts, artifacts, pending (add unlocks, remove rulesout), completed, currentposition
- Calls check_success() → sets solved=True if goal condition met

Environment: KNOWLEDGEVAULTPATH (default: /app/knowledge-vault)

5.2 GraphResolver

File: apps/conductor/graph_resolver.py

```
def resolve_branches(
    finding_type: str,
    satisfied_preconditions: set[str] | None = None,
    enrich_from_manifest: bool = True,
) -> list[BranchSpec]
```

Input
Output
Reads

BranchSpec fields

Field	Type	Description
concept_id	str	e.g. CONCEPT-CREDENTIAL-BRUTE
engine_id	`str | None`	
executable	bool	False = documented but no implementation
grows_from	list[str]	trigger finding_types
leaf_type	str	"terminal" · "canbeblocked" · "unknown"
stem_engines	list[dict]	Phase 4 fallback engines
preconditions	tuple	position/goal preconditions

5.3 LeafEvaluator

File: apps/conductor/leaf_evaluator.py

```
def evaluate_leaf(
    engine_id: str,
    concept_id: str,
    artifact: dict | None,
    state: str,
    finding_id: str = "",
) -> LeafClassification

def evaluate_verdict(
    engine_id: str,
    concept_id: str,
    artifact: dict | None,
    state: str,
    finding_id: str = "",
) -> Verdict
```

LeafClassification kinds: "terminal" · "blocked" · "none_yet" · "inconclusive"

Verdict fields: matchkind, goaladvanced, statechanges, artifactscaptured, unlocks, rules_out

Evaluation algorithm:

1. Fast path: issolved(artifact) → TERMINAL
2. RAG path: read leaf_type from graph
   - "terminal" → TERMINAL
   - "canbeblocked" + exhausted/timeout/failed → BLOCKED (grow stem)
   - Otherwise → INCONCLUSIVE

5.4 Planner

File: apps/conductor/planner.py

```
def plan_next_step(agent_state: dict) -> PlannerDecision
```

PlannerDecision fields: action, conceptid, engineid, priority, reason, log_entry

Action values: "execute" · "sweep" · "stuck" · "park" · "solved"

Algorithm:

1. If agent_state["solved"] → SOLVED
2. Filter pending − ruled_out − completed
3. Filter unmet preconditions (position + goal)
4. Reachable items → EXECUTE highest-priority
5. Only blocked items → SWEEP
6. Nothing → STUCK

---

6. Vault Pipeline

Environment variable: KNOWLEDGEVAULTPATH (all components)
Docker default: /app/knowledge-vault
Host default (new folder): /home/unctad/projects/conductor-framework/knowledge-vault

6.1 QwenClient

File: apps/conductor/qwen/client.py

```
async def chat(messages: list[dict], tools: list | None, timeout: int = 30) -> dict
async def ping() -> bool
async def ping_details() -> dict
```

Config
Model
Errors
ping()

6.2 ContextBuilder

File: apps/conductor/qwen/context_builder.py

```
async def build_investigation_context(
    investigation_id: str,
    target: str = "",
    max_findings: int = 25,
) -> str
```

Context sections (all graceful — section omitted on error):

Section	Source	Timeout
Findings Summary	validation.db normalized_findings	—
API Schema (V2.6)	GET /openapi.json on target	5 s
Product Catalog (V2.6)	GET /api/products on target (with session co	5 s
Vault Context (V2.7)	KNOWLEDGEVAULTPATH/findings/ + /labs/ t	—

6.3 VaultWriter

File: apps/conductor/qwen/vault_writer.py

```
async def export_finding(finding_id, engine_id, finding, artifact, target, investigation_id) -> str | ...
async def export_lab_walkthrough(investigation_id, target, findings, artifact) -> str | None
async def export_failure(investigation_id, engine_id, finding, exhaustion, error) -> str | None
```

Method	Writes to	Format
export_finding	KNOWLEDGEVAULTPATH/findings/web/FIND-	OWASP/MITRE tagged markdown
exportlabwalkthrough	KNOWLEDGEVAULTPATH/labs/web/LAB-WEB	Lab walkthrough
export_failure	KNOWLEDGEVAULTPATH/findings/web/FAIL-	Failure + exhaustion record

Threading: all operations fire-and-forget via asyncio.create_task().
Sequential ID: seqlock (threading.Lock) ensures no ID collisions.
Never raises — errors logged silently.

6.4 VaultEmbedder

File: apps/conductor/qwen/vault_embedder.py

```
async def rebuild_index() -> None
async def search_by_tags(tags: list[str], top_k: int = 10) -> list[dict]
async def search_semantic(query: str, top_k: int = 10) -> list[dict]
```

Fallback chain:

```
ChromaDB + Ollama nomic-embed-text
    ↓ (unavailable)
TF-IDF similarity
    ↓ (error)
Empty results []
```

rebuild_index() scans: concepts/ · findings/web/ · labs/web/
search_semantic() applies reranker (qwen3-reranker:0.6B if in Ollama registry; fallback to embedding order).

---

7. Export Layer

7.1 VaultManager

File: apps/investigationcore/exports/vault/vaultmanager.py

```
class VaultManager:
    @property
    def root(self) -> Path

    def investigation_dir(investigation_id: str) -> Path
    def investigation_report_path(investigation_id: str, target: str) -> Path
    def ensure_investigation_dir(investigation_id: str) -> Path
    def campaign_dir(campaign_id: str) -> Path
    def campaign_synthesis_path(campaign_id: str, title: str) -> Path
    def ensure_campaign_dir(campaign_id: str) -> Path
    def vault_relative(absolute: Path) -> str
```

Path resolution:

- Root: KNOWLEDGEVAULTPATH env var or <repo_root>/knowledge-vault

- Investigation dir: {root}/investigations/{id[:8]}/
- Campaign dir: {root}/campaigns/{id[:8]}/

7.2 ExportService (investigation_core)

File: apps/investigationcore/exports/services/exportservice.py

```
async def generate_export(
    investigation_id: str,
    target: str,
    findings: list[Any],
    evidence: list[Any],
    guided_action_objects: list[Any] | None = None,
) -> ExportRecord
```

Reads: orchestration.db, timeline events, mastery records, knowledge vault
Writes: VaultManager investigation_dir (markdown files)
Returns: ExportRecord with file paths + errors list

7.3 ExporterService (exporters)

File: exporters/exporter_service.py

```
class ExportBatch:
    findings: list[Finding]
    hosts: dict[str, Host]
    services: dict[str, Service]
    cves: dict[str, CVE]

class ExportReport:
    findings_written: list[Path]
    remediations_written: list[Path]
    discovery_written: list[Path]
    chains_written: list[Path]
    errors: list[str]

class ExporterService:
    def __init__(self, vault_path: Path | None = None) -> None
    def export_findings(batch: ExportBatch, export_remediations: bool = True) -> ExportReport
```

---

8. Seam Inventory

All seams defined in docs/architecture/seams.md. Regression tests in tests/conductor/_patterns/.

#	Contract	Guard	Pattern test
1	Finding ID round-trip: evt.id → nor	Explicit finding_id=evt.id pass	testseamidentity_roundtrip
2	Finding ID determinism: `SHA256(	target	path
3	DB target: FilterOrchestrator write	Lazy DB_PATH import inside closure	testseamdbtargetconsistency
4	Attribution: probeid → sourcescan	StorageBridge reads finding dict	testseamattribution_preservation
5	Gate registry: engine.gateid must	engineisvisible() fail-closed	testseamadapter_contract
6	RuleResult protocol: matched, stat	RuleResult dataclass	testseamrulechainevidence_gap
7	Wordlist registry: engines use loac	WordlistNotFoundError on unknown ID	(integration coverage)
8	OutputChannel: engine output thr	Single ingress principle	testseamoutput_channel
9	Watchdog: threshold from resourc	Threshold selection in _launch()	testseamresource_policy
10	Event bus: orchestrators emit via	EventBus singleton	(integration coverage)
11	Audit log: events → orchestration_	NOT YET WIRED — deferred	(deferred)
12	Filter→Exploit: only validated/incor	hasv2_engine filter in topology	testseamv2enumtoexploitinput
13	Branch persistence: deterministic	exploitation_branches table	testseamv2branchpersistence
14	Exhaustion handoff: ExhaustionRe	Schema in leaf_evaluator.py	testseamv2exhaustionrecord
15	Session pipe: per-investigation Se	INVESTIGATIONIDCTX in runtime.py	testseamsession_pipe
16	Re-invocation stub: extendedpaylo	All engines accept stubs	testseamv2reinvocationhook
17	Feedback hook: templatespec.feed	V2.1 renders control	testseamv2feedbackhook
18	RAG injection: injectragcontext(fin	Pure function, no side effects	testseamv2raginjection
19	Bootstrap phase state: PhasedRun	isexploitationactive() predicate	testseambootstrapphasestate
20	Engine auth write-back: outputcha	Credential engines call store_auth	testseamengineauthwrite
21	MSF recipe lookup: service_name	lookuprecipekey() pure function	testseammsfrecipelookup
22	Exploitation mindmap panel: _brar	No secondary server fetch in panel	testseampaneldatasource
23	Engine topology registration: Sugg	Execute route calls dispatch_engine	testseamv2enginetopology_registration
24	Conductor ID alignment: startinve	active[investigationid] keyed correctly	testseamconductorworkspaceid_alignment

9. Input / Output Summary

Layer 3

Compone	Inputs	Outputs	Reads	Writes
Conductor	target, config, investigation_id	investigation_id (str)	orchestration.db	orchestration.db (investigations)
_Investiga	investigationid, target, config,	context object	—	wires 6 orchestrators

Layer 2

Compone	Inputs	Outputs	Reads	Writes
PhaseOrch	—	phase name or None	phase state	orchestration.db phase state
Execution(	engine_id, target, config	run_id (str)	engine registry	_running dict
OutputCha	finding dict, target, status, aut	—	session_store	FindingsOrch, SessionOrch, BranchWriter
FindingsOr	finding dict, engine_id	finding_id or None	findings store	deduplicated findings, events
ChainOrch	finding dict	—	next_actions	chain events, orchestration.db
SessionOr	target, engine_caps, auth dict	(bool, float)	apiauth_store	apiauth_store
FilterOrche	—	—	validation.db normalized_findi	validation.db (validationstatus, blockreason)

Layer 1

Compone	Inputs	Outputs	Reads	Writes
Engine.run	target, config, sessionstore, o	EngineResult	session_store, external APIs	output_channel (findings, branches, auth)

Vault Pipeline

Compone	Inputs	Outputs	Reads	Writes
QwenClien	messages, tools, timeout	response dict	Ollama API	—
ContextBu	investigation_id, target	context string	validation.db, target APIs, vau	—
VaultWrite	finding/artifact data	file path or None	engine metadata	KNOWLEDGEVAULTPATH/findings/ labs/
VaultEmbe	tags or query, top_k	list[dict]	vault .md files, ChromaDB	ChromaDB (on rebuild)

---

10. Database Map

Database	Location	Owner	Tables (key)
validation.db	data/validation.db	apps.validation	normalizedfindings, exploitationbranches, investigat...
orchestration	data/orchestration.db	Conductor	investigations, investigationstate, orchestrationeve...

Seam 3 invariant: FilterOrchestrator DB writers must target validation.db. Lazy import inside closure prevents accidental orchestration.db capture.

Seam 13 invariant: exploitationbranches lives in validation.db. Written by BranchWriter (wired in ExecutionOrchestrator.init_). Branch state transitions unidirectional.

Generated 2026-06-04 from conductor-framework productization fork.